Laporan Tugas Kecil 1 IF2211 Strategi Algoritma

Semester II Tahun 2025/2026

Penyelesaian Permainan ***Queens*** Linkedin

Oleh :

Agatha Tatianingseto

13524008

INSTITUT TEKNOLOGI BANDUNG

Jalan Ganesha No. 10, Bandung, Jawa Barat

2025

Daftar Isi

1. Analisis Permasalahan dan Algoritma Brute Force

1.1. Analisis Permainan Queens dan Alur Berpikir

Permainan Queens merupakan pengembangan dari permasalahan klasik N-Queens, dengan tambahan batasan berupa wilayah berwarna serta larangan penempatan queen yang bersebelahan, termasuk secara diagonal. Sehingga solusi yang valid harus memenuhi seluruh kondisi secara bersamaan :

- Setiap baris harus mengandung tepat 1 queen
- Setiap kolom harus mengandung tepat 1 queen
- Setiap wilayah warna harus mengandung tepat 1 queen
- Tidak ada 2 queen yang berada pada kotak bersebelahan (baik secara horizontal, vertikal, maupun diagonal)

Dari keempat syarat tersebut, terlihat bahwa:

- Seluruh syarat bersifat global, artinya validitas suatu solusi baru dapat dipastikan setelah seluruh queen ditempatkan
- Tidak terdapat aturan lokal yang secara pasti dapat menjamin bahwa suatu penempatan parsial akan mengarah pada solusi yang valid
- Setiap kemungkinan penempatan queen pada satu berpotensi menjadi bagian dari solusi, meskipun pada akhirnya dapat gagal ketika seluruh papan diperiksa

Oleh karena itu, pendekatan yang paling langsung adalah dengan mencoba seluruh kemungkinan penempatan queen, kemudian memeriksa validitasnya secara keseluruhan.

1.2. Algoritma Brute Force yang digunakan

Algoritma yang digunakan bertujuan untuk mengenumerasi seluruh kemungkinan konfigurasi penempatan queen, lalu memeriksa apakah konfigurasi tersebut memenuhi seluruh aturan permainan. Secara umum, algoritma bekerja dengan cara menempatkan satu queen pada setiap baris, dengan mencoba semua kemungkinan kolom pada baris tersebut, hingga seluruh baris terisi.

1.2.1. Representasi Solusi

Solusi direpresentasikan sebagai sebuah array satu dimensi:

- solusi[i] = j menyatakan bahwa queen pada baris ke-i ditempatkan di kolom ke-j

Dengan representasi ini :

- Jumlah kemungkinan konfigurasi adalah N^N
- Setiap baris selalu memiliki tepat satu queen

Pseudocode :

```
buat array solusi berukuran N
```

```
untuk setiap i dari 0 sampai N-1:
solusi[i] = 0
```

1.2.2. Enumerasi Semua Kemungkinan Penempatan

Algoritma melakukan enumerasi secara rekursif, dengan mencoba seluruh kemungkinan kolom pada setiap baris. Pada tahap ini belum dilakukan pengecekan terhadap aturan apapun, sehingga penempatan Queen

- Boleh berada pada kolom yang sama
- Boleh berada pada warna yang sama
- Boleh bersebelahan

Hal ini dilakukan untuk memastikan bahwa algoritma benar-benar mencoba semua konfigurasi yang mungkin

Pseudocode :

```
procedure ENUMERATE(row):
jika row == N:
periksa apakah solusi valid
                return
  untuk col dari 0 sampai N-1:
     solusi[row] = col
     ENUMERATE(row + 1)
```

1.2.3. Pemeriksaan Validitas Solusi

Ketika seluruh baris telah diisi (row == N), algoritma melakukan pemeriksaan terhadap konfigurasi yang terbentuk.

Pemeriksaan dilakukan terhadap:

- Keunikan kolom
- Keunikan warna
- Tidak ada queen yang saling bersebelahan

Jika salah satu syarat tidak terpenuhi, konfigurasi ditolak dan mulai pencarian pada kemungkinan selanjutnya

Pseudocode :

```
procedure CEK_SOLUSI(solusi):
    jika ada dua queen di kolom yang sama:
        return false
    jika ada dua queen di region yang sama:
        return false
    jika ada dua queen bersebelahan (termasuk diagonal):
        return false
    return true
```

1.2.4. Penyimpanan Solusi dan Penghentian Pencarian

Jika sebuah konfigurasi dinyatakan valid :

- Konfigurasi tersebut disalin sebagai solusi akhir
- Proses pencarian dihentikan
- Jumlah iterasi yang telah diperiksa dicatat

Pseudocode:

```
jika CEK_SOLUSI(solusi) == true:
    simpan solusi
    hentikan pencarian
```

1.3. Alasan Algoritma Termasuk Pure Brute Force

Algoritma yang digunakan termasuk pure brute force karena :

1. Seluruh kemungkinan konfigurasi dieksplorasi

Jumlah konfigurasi yang dicoba adalah N^N, sesuai dengan seluruh kemungkinan penempatan queen per baris

2. Tidak ada pruning atau optimasi awal

   Tidak ada pengecekan kolom, warna, atau adjacency selama proses enumerasi

3. Validasi hanya dilakukan pada solusi lengkap

   Pemeriksaan aturan dilakukan setelah seluruh queen ditempatkan, bukan selama proses pembentukan solusi

4. Tidak menggunakan heuristik

   Algoritma tidak menggunakan informasi tambahan untuk mengurangi ruang pencarian

2. Source Program (Go)

2.1. algo.go – Algoritma Pencarian Solusi

Mengimplementasikan proses enumerasi kandidat solusi dan validasi solusi

2.1.1. type solve

Struktur hasil pencarian solusi yang menyimpan status ditemukan/tidak (ada), array posisi queen per baris (hasil), dan jumlah konfigurasi yang telah diperiksa (iterasi).

```
type solve struct {
        ada     bool
        hasil   []int
        iterasi int64
}
```

2.1.2. type trace_live

Tipe fungsi callback untuk *live tracing* yang dipanggil ketika satu kandidat solusi lengkap terbentuk.

```
type trace_live func(b *Queen, solusi []int, iterasi int64)
```

2.1.3. find_solusi(b *Queen, live trace_live) solve

Melakukan enumerasi semua kemungkinan penempatan queen dengan merepresentasikan kandidat sebagai array solusi[row] = col. Validasi hanya dilakukan saat kandidat lengkap (row == N), dan proses berhenti ketika solusi pertama ditemukan.

```
func find_solusi(b *Queen, live trace_live) solve {
	N := b.n

	// solusi = array untuk simpan percobaan kemungkinan
	solusi := make([]int, N)
	for i := 0; i < N; i++ {
		solusi[i] = 0
	}

	var iterasi int64 = 0
	found := false

	// final = hasil copy dari array solusi kalau kemungkinan sudah valid
	final := make([]int, N)
	for i := 0; i < N; i++ {
		final[i] = -1
	}

	// cari semua kemungkinan solusi, stop kalau solusi udah follow semua rules
	var algo func(row int)
	algo = func(row int) {
		if found {
			return
		}

		if row == N {
			iterasi++

			if live != nil {
				live(b, solusi, iterasi)
			}

			if cek_solusi(b, solusi) {
				found = true
				for i := 0; i < N; i++ {
					final[i] = solusi[i]
				}
			}
			return
		}

		for col := 0; col < N; col++ {
			solusi[row] = col
			algo(row + 1)
			if found {
				return
			}
		}
	}

	algo(0)

	return solve{
```

```
                ada:    found,
                hasil:  final,
                iterasi: iterasi,
        }
}
```

2.1.4. cek_solusi(b *Queen, solusi []int) bool

Memeriksa apakah kandidat memenuhi seluruh aturan: kolom unik (set boolean), region unik (set boolean berdasarkan b.id), serta tidak ada queen yang bersebelahan (cek adjacency 8 arah menggunakan matriks used).

```
func cek_solusi(b *Queen, solusi []int) bool {
        N := b.n

        usedColumn := make([]bool, N)
        for i  := 0; i  < N; i++ {
                c := solusi[i]
                if c < 0 || c >= N {
                        return false
                }
                if usedColumn[c] {
                        return false
                }
                usedColumn[c] = true
        }

        usedColor := make([]bool, b.idUnique)
        for i  := 0; i  < N; i++ {
                c := solusi[i]
                color := b.id[i][c]
                if color < 0 || color >= b.idUnique {
                        return false
                }
                if usedColor[color] {
                        return false
                }
                usedColor[color] = true
        }

        used := make([][]bool, N)
        for i := 0; i < N; i++ {
                used[i] = make([]bool, N)
        }
        for j := 0; j < N; j++ {
                used[j][solusi[j]] = true
        }

        // cek supaya solusi tidak bersebelahan dan tidak diagonal
        for k := 0; k < N; k++ {
```

```
                c := solusi[k]
                for dr := -1; dr <= 1; dr++ {
                        for dc := -1; dc <= 1; dc++ {
                                if dr == 0 && dc == 0 {
                                        continue
                                }
                                nr := k + dr
                                nc := c + dc
                                if nr < 0 || nr >= N || nc < 0 || nc >= N {
                                        continue
                                }
                                if used[nr][nc] {
                                        return false
                                }
                        }
                }
        }

        return true
}
```

2.1.5. total_configurations(N int) int64

Menghitung jumlah konfigurasi teoretis yang mungkin pada representasi row→ col yaitu N^N.

```
func total_configurations(N int) int64 {
        return int64(math.Pow(float64(N), float64(N)))
}
```

2.2. parser.go – Parser Input dan Validasi

Membaca file input .txt, mengubahnya menjadi struktur data Queen, dan memastikan input valid sesuai aturan permainan

2.2.1. type Queen

Struktur board: n ukuran papan, raw karakter asli per sel, id integer ID region per sel, idUnique jumlah region unik.

```
type Queen struct {
        n int
        id [][]int
        idUnique int
        raw [][]rune
}
```

2.2.2. parse_board_from_file(path string) (*Queen, error)

Membaca file input menggunakan scanner, menyimpan seluruh baris, lalu membentuk board melalui parse_lines.

```
func parse_board_from_file(path string) (*Queen, error) {
	f, err := os.Open(path)
	if err != nil {
		return nil, fmt.Errorf("Failed to Open FIle %q: %w", path, err)
	}
	defer f.Close()

	sc := bufio.NewScanner(f)
	buff := make([]byte, 0, 64*1024)
	sc.Buffer(buff, 1024*1024)

	var lines []string
	for sc.Scan() {
		lines = append(lines, sc.Text())
	}

	if err := sc.Err(); err!=nil {
		return nil, fmt.Errorf("failed reading file %q: %w", path, err)
	}

	return parse_lines(lines)
}
```

2.2.3. read_line(lines []string) []string

Membersihkan input: menghapus baris kosong dan melakukan trimming.

```
func read_line(lines []string) []string {
	results := make([]string, 0, len(lines))
	for i := 0; i < len(lines); i++ {
		line := lines[i]
		t := strings.TrimSpace(line)
		if t == "" {
			continue
		}
		results = append(results, t)
	}
	return results
}
```

2.2.4. parse_lines(lines []string) (*Queen, error)

Mengubah baris input menjadi board NxN: menghapus whitespace per baris, memastikan dimensi konsisten, serta memetakan karakter region menjadi ID integer (idMap).

```
func parse_lines(lines []string) (*Queen, error) {
	lines = read_line(lines)
	N := len(lines)
	if N == 0 {
		return nil, fmt.Errorf("no lines found")
	}

	raw := make([][]rune, N)
	id := make([][]int, N)

	idMap := make(map[rune]int)
	nextID := 0

	for i := 0; i<N; i++ {
		temp := make([]rune, 0, N)
		runes := []rune(lines[i])
		for j := 0; j<len(runes); j++ {
			ch := runes[j]
			if unicode.IsSpace(ch){
				continue
			}
			temp = append(temp, ch)
		}

		if len(temp) != N {
			return nil, fmt.Errorf("different dimension: row %d has %d
cells, expected %d (board must be NxN)", i+1, len(temp), N,)
		}

		raw[i] = make([]rune, N)
		id[i] = make([]int, N)

		for k := 0; k < N; k++ {
			a := temp[k]
			raw[i][k] = a

			b, ok := idMap[a]
			if !ok {
				idMap[a] = nextID
				b = nextID
				nextID++
			}
			id[i][k] = b
		}
	}

	return &Queen{
		n: N,
		id: id,
		idUnique: nextID,
```

```
                raw: raw,
        }, nil
}
```

2.2.5. validate_board(b *Queen) error

Memvalidasi board: memastikan ukuran NxN konsisten, idUnique valid, jumlah region unik harus sama dengan N, nilai region ID tiap sel valid, serta mapping karakter↔ID konsisten.

```
func validate_board(b *Queen) error {
        if b == nil {
                return fmt.Errorf("board is nil")
        }
        if b.n <= 0 {
                return fmt.Errorf("invalid board size n=%d", b.n)
        }

        // cek dimensi
        if len(b.raw) != b.n {
                return fmt.Errorf("invalid raw rows: got %d, expected %d", len(b.raw),
b.n)
        }
        if len(b.id) != b.n {
                return fmt.Errorf("invalid id rows: got %d, expected %d", len(b.id), b.n)
        }

        // cek jumlah column - panjang row
        for r := 0; r < b.n; r++ {
                if len(b.raw[r]) != b.n {
                        return fmt.Errorf("invalid raw row %d length: got %d, expected
%d", r+1, len(b.raw[r]), b.n)
                }
                if len(b.id[r]) != b.n {
                        return fmt.Errorf("invalid id row %d length: got %d, expected
%d", r+1, len(b.id[r]), b.n)
                }
        }

        if b.idUnique <= 0 {
                return fmt.Errorf("invalid region count: %d", b.idUnique)
        }
        if b.idUnique != b.n {
                return fmt.Errorf(
                        "invalid board: number of regions (%d) must equal N (%d)
because there must be exactly one queen per row/col/region",
                        b.idUnique, b.n,
                )
        }
```

```
	for r := 0; r < b.n; r++ {
		for c := 0; c < b.n; c++ {
			v := b.id[r][c]
			if v < 0 || v >= b.idUnique {
				return fmt.Errorf("invalid region id at (%d,%d): got %d,
expected 0..%d",
					r+1, c+1, v, b.idUnique-1)
			}
		}
	}

	// cek raw dan id, apakah konsisten?
	temp := make(map[rune]int)
	for i := 0; i < b.n; i++ {
		for j := 0; j < b.n; j++ {
			ch := b.raw[i][j]
			v := b.id[i][j]
			if old, ok := temp[ch]; ok {
				if old != v {
					return fmt.Errorf("inconsistent mapping for
symbol %q: previously %d, now %d at (%d,%d)",
						ch, old, v, i+1, j+1)
				}
			} else {
				temp[ch] = v
			}
		}
	}

	return nil
}
```

2.3. render.go – Output Text, Save .txt, dan Live Report

Menyediakan output board dalam bentuk plain ASCII (tanpa ANSI), menyimpan file solusi dan menulis live report

2.3.1. repeatPlain(s string, n int) string

Utility pengulangan string untuk membentuk border grid pada output file.

```
func repeatPlain(s string, n int) string {
	if n <= 0 {
		return ""
	}
	out := ""
	for i := 0; i < n; i++ {
		out += s
	}
	return out
```

```
}
```

2.3.2. build_grid_lines_plain(b *Queen, solusi []int) ([]string, error)

Membangun grid ASCII lengkap (border + separator) dalam bentuk array string. Jika solusi diberikan, queen ditampilkan sebagai #. Fungsi ini dipakai untuk output file .txt dan live report agar tidak ada karakter “aneh”.

```
func build_grid_lines_plain(b *Queen, solusi []int) ([]string, error) {
	N := b.n
	if N <= 0 {
		return nil, fmt.Errorf("invalid board size: %d", N)
	}
	if solusi != nil && len(solusi) != N {
		return nil, fmt.Errorf("invalid solution length: got %d, expected %d",
len(solusi), N)
	}

	q := make([]int, N)
	for i := 0; i < N; i++ {
		q[i] = -1
	}
	if solusi != nil {
		for r := 0; r < N; r++ {
			q[r] = solusi[r]
			if q[r] < 0 || q[r] >= N {
				return nil, fmt.Errorf("invalid queen column at row %d:
%d", r+1, q[r])
			}
		}
	}

	cellW := 3
	lines := make([]string, 0, N*2+1)

	top := " ┌" + repeatPlain("─", cellW)
	for i := 1; i < N; i++ {
		top += "┬" + repeatPlain("─", cellW)
	}
	top += "┐ "
	lines = append(lines, top)

	for r := 0; r < N; r++ {
		row := " │ "
		for c := 0; c < N; c++ {
			ch := b.raw[r][c]
			if q[r] == c {
				ch = '#'
			}
			row += " " + string(ch) + " "
```

```
                        if c != N-1 {
                                row += " │ "
                        }
                }
                row += " │ "
                lines = append(lines, row)

                if r != N-1 {
                        mid := " ├" + repeatPlain("─", cellW)
                        for i := 1; i < N; i++ {
                                mid += "┼" + repeatPlain("─", cellW)
                        }
                        mid += "┤ "
                        lines = append(lines, mid)
                }
        }

        bot := " └" + repeatPlain("─", cellW)
        for i := 1; i < N; i++ {
                bot += "┴" + repeatPlain("─", cellW)
        }
        bot += "┘ "
        lines = append(lines, bot)

        return lines, nil
}
```

2.3.3. type liveReport

Menyimpan writer file, interval snapshot, dan waktu snapshot terakhir.

```
type liveReport struct {
        w        *bufio.Writer
        f        *os.File
        interval time.Duration
        last     time.Time
}
```

2.3.4. live_writer(path string, interval time.Duration) (*liveReport, error)

Membuat file trace dan menyiapkan buffered writer.

```
func live_writer(path string, interval time.Duration) (*liveReport, error) {
        f, err := os.Create(path)
        if err != nil {
                return nil, fmt.Errorf("failed to create trace file %q: %w", path, err)
        }
```

```
	return &liveReport{
		w:	bufio.NewWriter(f),
		f:	f,
		interval: interval,
		last:	time.Time{},
	}, nil
}
```

2.3.5. (tw *liveReport) Close() error

Flush buffer dan menutup file trace.

```
func (tw *liveReport) Close() error {
	if tw == nil {
		return nil
	}
	if err := tw.w.Flush(); err != nil {
		_ = tw.f.Close()
		return err
	}
	return tw.f.Close()
}
```

2.3.6. (tw *liveReport) writer_snapshot(b *Queen, solusi []int, iterasi int64)

Menulis snapshot kandidat solusi pada interval tertentu: mencetak iterasi, kandidat row→col, dan grid board ASCII (tanpa ANSI), lalu separator "----".

```
func (tw *liveReport) writer_snapshot(b *Queen, solusi []int, iterasi int64) {
	if tw == nil {
		return
	}
	now := time.Now()
	if !tw.last.IsZero() && now.Sub(tw.last) < tw.interval {
		return
	}
	tw.last = now

	grid, err := build_grid_lines_plain(b, solusi)
	if err != nil {
		return
	}

	_, _ = tw.w.WriteString(fmt.Sprintf("ITERASI: %d\n", iterasi))
	_, _ = tw.w.WriteString(fmt.Sprintf("KANDIDAT (row->col): %v\n", solusi))
	for i := 0; i < len(grid); i++ {
		_, _ = tw.w.WriteString(grid[i] + "\n")
```

```
        }
        _, _ = tw.w.WriteString("----\n")
        _ = tw.w.Flush()
}
```

2.3.7. write_lines_to_file(outPath string, lines []string, execMs int64, iterasi int64) error

Menyimpan output teks (grid/plain) ke file, serta menambahkan informasi waktu dan jumlah iterasi.

```
func write_lines_to_file(outPath string, lines []string, execMs int64, iterasi int64) error {
        f, err := os.Create(outPath)
        if err != nil {
                return fmt.Errorf("failed to create output file %q: %w", outPath, err)
        }
        defer f.Close()

        w := bufio.NewWriter(f)
        for i := 0; i < len(lines); i++ {
                _, _ = w.WriteString(lines[i] + "\n")
        }
        _, _ = w.WriteString(fmt.Sprintf("Waktu pencarian: %d ms\n", execMs))
        _, _ = w.WriteString(fmt.Sprintf("Banyak kasus yang ditinjau: %d kasus\n",
iterasi))

        if err := w.Flush(); err != nil {
                return fmt.Errorf("failed to write output file %q: %w", outPath, err)
        }
        return nil
}
```

2.4. render_image.go – Output PNG

Membuat gambar PNG dari Queen, memberi warna berdasarkan wilayah, dan menambahkan ikon crown pada posisi Queen

2.4.1. save_gambar(b *Queen, solusi []int, crownPath, outPath string, cellSize, tebalGrid, outerBorder int) error

Menggambar board ke canvas: mewarnai sel berdasarkan region, overlay crown (jika ada), menggambar grid/border, lalu menyimpan menjadi PNG.

```
unc save_gambar(b *Queen, solusi []int, crownPath, outPath string, cellSize, tebalGrid,
outerBorder int) error {
        N := b.n
        if N <= 0 {
```

```
                return fmt.Errorf("invalid board size: %d", N)
        }
        if len(solusi) != N {
                return fmt.Errorf("invalid solution length: got %d, expected %d",
len(solusi), N)
        }

        w := outerBorder*2 + N*cellSize
        h := outerBorder*2 + N*cellSize

        img := image.NewRGBA(image.Rect(0, 0, w, h))

        draw.Draw(img, img.Bounds(), &image.Uniform{color.RGBA{255, 255, 255,
255}}, image.Point{}, draw.Src)

        warna := generate_warna(b.idUnique)

        for r := 0; r < N; r++ {
                for c := 0; c < N; c++ {
                        reg := b.id[r][c]
                        if reg < 0 || reg >= b.idUnique {
                                return fmt.Errorf("invalid region id at (%d,%d): %d",
r+1, c+1, reg)
                        }

                        x0 := outerBorder + c*cellSize
                        y0 := outerBorder + r*cellSize
                        cellRect := image.Rect(x0, y0, x0+cellSize, y0+cellSize)

                        draw.Draw(img, cellRect, &image.Uniform{warna[reg]},
image.Point{}, draw.Src)
                }
        }

        var crown image.Image
        if crownPath != "" {
                cr, err := loadPNG(crownPath)
                if err != nil {
                        return fmt.Errorf("failed to load crown asset: %w", err)
                }
                crown = cr
        }

        if crown != nil {
                targetW := int(float64(cellSize) * 0.70)
                targetH := int(float64(cellSize) * 0.70)
                if targetW < 1 {
                        targetW = 1
                }
                if targetH < 1 {
                        targetH = 1
                }
```

```
                scaledCrown := resizeNearest(crown, targetW, targetH)

                for r := 0; r < N; r++ {
                        c := solusi[r]
                        if c < 0 || c >= N {
                                return fmt.Errorf("invalid queen col at row %d: %d",
r+1, c)
                        }

                        x0 := outerBorder + c*cellSize
                        y0 := outerBorder + r*cellSize

                        cx := x0 + (cellSize-targetW)/2
                        cy := y0 + (cellSize-targetH)/2

                        dstRect := image.Rect(cx, cy, cx+targetW, cy+targetH)
                        draw.Draw(img, dstRect, scaledCrown, image.Point{},
draw.Over)
                }
        }

        gridColor := color.RGBA{0, 0, 0, 255}

        drawBorder(img, outerBorder, gridColor)

        if tebalGrid < 1 {
                tebalGrid = 1
        }

        for i := 1; i < N; i++ {
                x := outerBorder + i*cellSize
                drawVLine(img, x, outerBorder, outerBorder+N*cellSize, tebalGrid,
gridColor)

                y := outerBorder + i*cellSize
                drawHLine(img, y, outerBorder, outerBorder+N*cellSize, tebalGrid,
gridColor)
        }

        if err := savePNG(outPath, img); err != nil {
                return err
        }
        return nil
}
```

2.4.2. loadPNG(path string) (image.Image, error) / savePNG(path string, img image.Image) error

Fungsi helper untuk membaca/menyimpan file PNG.

```
func loadPNG(path string) (image.Image, error) {
```

```
	f, err := os.Open(path)
	if err != nil {
		return nil, fmt.Errorf("open %q: %w", path, err)
	}
	defer f.Close()

	img, err := png.Decode(f)
	if err != nil {
		return nil, fmt.Errorf("decode png %q: %w", path, err)
	}
	return img, nil
}
```

2.4.3. drawBorder(...), drawVLine(...), drawHLine(...)

Menggambar border luar dan garis grid pada gambar.

```
func drawBorder(img *image.RGBA, thickness int, col color.Color) {
	b := img.Bounds()
	// Top
	draw.Draw(img, image.Rect(b.Min.X, b.Min.Y, b.Max.X, b.Min.Y+thickness),
&image.Uniform{col}, image.Point{}, draw.Src)
	// Bottom
	draw.Draw(img, image.Rect(b.Min.X, b.Max.Y-thickness, b.Max.X, b.Max.Y),
&image.Uniform{col}, image.Point{}, draw.Src)
	// Left
	draw.Draw(img, image.Rect(b.Min.X, b.Min.Y, b.Min.X+thickness, b.Max.Y),
&image.Uniform{col}, image.Point{}, draw.Src)
	// Right
	draw.Draw(img, image.Rect(b.Max.X-thickness, b.Min.Y, b.Max.X, b.Max.Y),
&image.Uniform{col}, image.Point{}, draw.Src)
}
```

2.4.4. generate_warna(count int) []color.RGBA

Menghasilkan palet warna pastel sebanyak jumlah region menggunakan pendekatan HSV.

```
func generate_warna(count int) []color.RGBA {
	if count < 1 {
		return []color.RGBA{{200, 200, 200, 255}}
	}

	cols := make([]color.RGBA, count)
	for i := 0; i < count; i++ {
		h := float64(i) / float64(count) // 0..1
		cols[i] = hsv_to_rgb(h, 0.35, 0.95) // pastel-like
	}
```

```
	return cols
}
```

2.4.5. hsv_to_rgb(h,s,v), clamp01(x)

Konversi warna HSV→RGB dan pembatasan nilai agar tetap pada rentang valid.

```
func hsv_to_rgb(h, s, v float64) color.RGBA {
	h = math.Mod(h, 1.0)
	if h < 0 {
		h += 1.0
	}

	c := v * s
	x := c * (1 - math.Abs(math.Mod(h*6, 2)-1))
	m := v - c

	var r, g, b float64
	switch {
	case 0 <= h && h < 1.0/6.0:
		r, g, b = c, x, 0
	case 1.0/6.0 <= h && h < 2.0/6.0:
		r, g, b = x, c, 0
	case 2.0/6.0 <= h && h < 3.0/6.0:
		r, g, b = 0, c, x
	case 3.0/6.0 <= h && h < 4.0/6.0:
		r, g, b = 0, x, c
	case 4.0/6.0 <= h && h < 5.0/6.0:
		r, g, b = x, 0, c
	default:
		r, g, b = c, 0, x
	}

	R := uint8(clamp01(r+m) * 255)
	G := uint8(clamp01(g+m) * 255)
	B := uint8(clamp01(b+m) * 255)
	return color.RGBA{R, G, B, 255}
}
```

2.4.6. resizeNearest(src, newW, newH)

Mengubah ukuran gambar crown dengan metode nearest neighbor agar sederhana dan cepat.

```
func resizeNearest(src image.Image, newW, newH int) *image.RGBA {
	dst := image.NewRGBA(image.Rect(0, 0, newW, newH))
	sb := src.Bounds()
```

```
	sw := sb.Dx()
	sh := sb.Dy()
	if sw <= 0 || sh <= 0 {
		return dst
	}

	for y := 0; y < newH; y++ {
		sy := sb.Min.Y + (y*sh)/newH
		for x := 0; x < newW; x++ {
			sx := sb.Min.X + (x*sw)/newW
			dst.Set(x, y, src.At(sx, sy))
		}
	}
	return dst
}
```

2.5. main.go

Mengatur keseluruhan alur program: input file, menampilkan board, memilih opsi output, menjalankan solver, menampilkan hasil, menyimpan output, dan mengulang program

2.5.1. main()

Menjalankan program dalam loop: memanggil runOnce, lalu menanyakan apakah user ingin mencari solusi puzzle lain atau keluar.

```
func main() {
	in := bufio.NewReader(os.Stdin)

	for {
		runOnce(in)

		fmt.Println()
		if !askYesNo(in, "Apakah ingin mencari solusi permainan Queen
lainnya? (Ya/Tidak) ") {
			cprintln("\x1b[90m", "Bye") // gray
			return
		}
		fmt.Println()
	}
}
```

2.5.2. runOnce(in *bufio.Reader)

Menjalankan satu sesi pemecahan: menampilkan header, meminta path input, parse & validasi board, menampilkan board input, menanyakan opsi simpan (trace/txt/png),

menyiapkan folder output, menjalankan brute force, menampilkan hasil, dan menyimpan file output sesuai pilihan.

```
func runOnce(in *bufio.Reader) {
	printBox("Game Queen", []string{
		"1) Menerima input sebuah directory dari .txt yang memuat sebuah board",
		"2) Program akan mencari solusi peletakan queen sesuai dengan rules",
		"3) Hasil dan live report dapat di simpan",
	})

	// Input path
	inputPath := promptInputPath(in)

	// Parse
	b, err := parse_board_from_file(inputPath)
	if err != nil {
		cprintf("\x1b[31m", "ERR: %v\n", err)
		return
	}

	// Validasi input
	if err := validate_board(b); err != nil {
		cprintf("\x1b[31m", "INVALID BOARD: %v\n", err)
		return
	}

	// print board
	fmt.Println()
	PrintBoardGrid("BOARD INPUT", b, nil)
	fmt.Println()

	// tanya simpan
	wantTrace := askYesNo(in, "Apakah Anda ingin menyimpan proses pencarian (live report)? (Ya/Tidak) ")
	wantTXT := askYesNo(in, "Apakah Anda ingin menyimpan solusi sebagai .txt? (Ya/Tidak) ")
	wantPNG := askYesNo(in, "Apakah Anda ingin menyimpan solusi sebagai gambar PNG? (Ya/Tidak) ")

	needOutDir := wantTrace || wantTXT || wantPNG
	outDir := ""
	if needOutDir {
		outDir = promptOutputDirAndEnsure(in)
		cprintf("\x1b[90m", "Output folder: %s\n", outDir)
	}

	// live report
	var tw *liveReport
	var hook trace_live

	if wantTrace {
		tracePath := filepath.Join(outDir, "trace.txt")
```

```
                tw, err = live_writer(tracePath, 50*time.Millisecond)
                if err != nil {
                        cprintf("\x1b[31m", "ERR: %v\n", err)
                        return
                }
                defer tw.Close()

                cprintf("\x1b[90m", "Tracing aktif: %s (snapshot tiap 50ms)\n",
tracePath)

                hook = func(b *Queen, solusi []int, iterasi int64) {
                        tw.writer_snapshot(b, solusi, iterasi)
                }
        }

        // solve
        cprintln("\x1b[33m", "Solving...") // yellow
        start := time.Now()
        res := find_solusi(b, hook)
        execMs := time.Since(start).Milliseconds()

        if !res.ada {
                cprintln("\x1b[31m", "NO SOLUTION") // red
                fmt.Printf("Waktu pencarian: %d ms\n", execMs)
                fmt.Printf("Banyak kasus yang ditinjau: %d kasus\n", res.iterasi)
                return
        }

        // print hasil
        fmt.Println()
        cprintln("\x1b[32m", "SOLUTION FOUND ") // green
        PrintBoardGrid("BOARD RESULT", b, res.hasil)
        fmt.Println()

        fmt.Printf("Waktu pencarian: %d ms\n", execMs)
        fmt.Printf("Banyak kasus yang ditinjau: %d kasus\n", res.iterasi)

        // save
        if wantTXT {
                gridLines, err := build_grid_lines_plain(b, res.hasil)
                if err != nil {
                        cprintf("\x1b[31m", "ERR: %v\n", err)
                        return
                }

                outTXT := filepath.Join(outDir, "solution.txt")
                if err := write_lines_to_file(outTXT, gridLines, execMs, res.iterasi); err
!= nil {
                        cprintf("\x1b[31m", "ERR: %v\n", err)
                        return
                }
                cprintf("\x1b[36m", "Solusi disimpan ke %s\n", outTXT)
        }
```

```
        if wantPNG {
                crownPath := filepath.FromSlash("src/assets/crown.png")
                outPNG := filepath.Join(outDir, "solution.png")

                cellSize := 48
                gridThickness := 2
                outerBorder := 4

                if _, err := os.Stat(crownPath); err != nil {
                        cprintf("\x1b[33m", "Warning: %s tidak ditemukan, PNG akan
dibuat tanpa crown.\n", crownPath)
                        crownPath = ""
                }

                if err := save_gambar(b, res.hasil, crownPath, outPNG, cellSize,
gridThickness, outerBorder); err != nil {
                        cprintf("\x1b[31m", "ERR: %v\n", err)
                        return
                }
                cprintf("\x1b[36m", "Gambar disimpan ke %s\n", outPNG) // cyan
        }
}
```

2.5.3. promptInputPath(in *bufio.Reader) string

Meminta user memasukkan path file input .txt, mendukung path relatif/absolut, merapikan path (Clean), dan memastikan file ada serta bukan folder.

```
func promptInputPath(in *bufio.Reader) string {
        for {
                fmt.Print("Masukkan path lengkap file input (.txt): ")
                s, _ := in.ReadString('\n')
                s = strings.TrimSpace(s)
                s = strings.Trim(s, `"'`)

                if s == "" {
                        fmt.Println("Path kosong. Coba lagi.")
                        continue
                }

                if strings.HasPrefix(s, "~") {
                        if home, err := os.UserHomeDir(); err == nil {
                                s = filepath.Join(home, strings.TrimPrefix(s, "~"))
                        }
                }

                abs := s
                if !filepath.IsAbs(abs) {
```

```
                if a, err := filepath.Abs(abs); err == nil {
                        abs = a
                }
        }
        abs = filepath.Clean(abs)

        info, err := os.Stat(abs)
        if err != nil {
                fmt.Println("File tidak ditemukan / tidak bisa diakses:", err)
                continue
        }
        if info.IsDir() {
                fmt.Println("Itu directory, bukan file. Masukkan path file .txt.")
                continue
        }

        return abs
    }
}
```

2.5.4. promptOutputDirAndEnsure(in *bufio.Reader) string

Meminta user memasukkan folder output dan membuat folder jika belum ada (os.MkdirAll), serta memastikan path tersebut benar-benar directory.

```
func promptOutputDirAndEnsure(in *bufio.Reader) string {
    for {
        fmt.Print("Masukkan directory/folder output untuk menyimpan file: ")
        s, _ := in.ReadString('\n')
        s = strings.TrimSpace(s)
        s = strings.Trim(s, `"'`)

        if s == "" {
                fmt.Println("Folder output kosong. Coba lagi.")
                continue
        }

        if strings.HasPrefix(s, "~") {
                if home, err := os.UserHomeDir(); err == nil {
                        s = filepath.Join(home, strings.TrimPrefix(s, "~"))
                }
        }

        abs := s
        if !filepath.IsAbs(abs) {
                if a, err := filepath.Abs(abs); err == nil {
                        abs = a
                }
        }
        abs = filepath.Clean(abs)
```

```
                if err := os.MkdirAll(abs, 0o755); err != nil {
                        fmt.Println("Gagal membuat folder output:", err)
                        continue
                }

                info, err := os.Stat(abs)
                if err != nil {
                        fmt.Println("Folder output tidak bisa diakses:", err)
                        continue
                }
                if !info.IsDir() {
                        fmt.Println("Path output bukan folder (ternyata file). Masukkan
folder yang benar.")
                        continue
                }

                return abs
        }
}
```

2.5.5. askYesNo(in *bufio.Reader, prompt string) bool

Utility input Ya/Tidak yang robust, menerima variasi jawaban (ya/y/yes, tidak/t/no/n).

```
func askYesNo(in *bufio.Reader, prompt string) bool {
        for {
                fmt.Print(prompt)
                s, _ := in.ReadString('\n')
                s = strings.ToLower(strings.TrimSpace(s))

                switch s {
                case "ya", "y", "yes":
                        return true
                case "tidak", "t", "no", "n":
                        return false
                default:
                        fmt.Println("Jawaban tidak valid. Ketik Ya/Tidak.")
                }
        }
}
```

2.6. cli.go – ASCII UI dan Warna Terminal

Mengimplementasikan utilitas pewarnaan ANSI dan tampilan board sebagai grid ASCII

ANSI

2.6.1. type ansi

Alias tipe string untuk menyimpan escape sequence ANSI.

```
type ansi string
```

2.6.2. const ansiReset, ansiBold

Kode ANSI untuk reset warna dan teks bold

```
const (
	ansiReset ansi = "\x1b[0m"
	ansiBold  ansi = "\x1b[1m"
)
```

2.6.3. regionFg

Daftar warna foreground yang digunakan untuk membedakan region.

```
var regionFg = []ansi{
	"\x1b[31m", // red
	"\x1b[32m", // green
	"\x1b[33m", // yellow
	"\x1b[34m", // blue
	"\x1b[35m", // magenta
	"\x1b[36m", // cyan
	"\x1b[91m", // bright red
	"\x1b[92m", // bright green
	"\x1b[93m", // bright yellow
	"\x1b[94m", // bright blue
	"\x1b[95m", // bright magenta
	"\x1b[96m", // bright cyan
}
```

2.6.4. colorsEnabled() bool

Menentukan apakah warna diaktifkan (warna dimatikan bila env NO_COLOR diset).

```
func colorsEnabled() bool {
	if os.Getenv("NO_COLOR") != "" {
		return false
	}
	return true
```

```
}
```

2.6.5. colorWrap(s string, codes ...ansi) string

Membungkus string dengan kode ANSI tertentu (warna/bold) dan menutupnya dengan reset.

```
func colorWrap(s string, codes ...ansi) string {
	if !colorsEnabled() || len(codes) == 0 {
		return s
	}
	var b strings.Builder
	for _, c := range codes {
		b.WriteString(string(c))
	}
	b.WriteString(s)
	b.WriteString(string(ansiReset))
	return b.String()
}
```

2.6.6. cprintln(c ansi, s string)

Mencetak 1 baris teks dengan warna tertentu.

```
func cprintln(c ansi, s string) {
	fmt.Println(colorWrap(s, c))
}
```

2.6.7. cprintf(c ansi, format string, a ...any)

Mencetak teks berformat dengan warna tertentu.

```
func cprintf(c ansi, format string, a ...any) {
	out := fmt.Sprintf(format, a...)
	fmt.Print(colorWrap(out, c))
}
```

ASCII

2.6.8. repeat(s string, n int) string

Mengulang string untuk membentuk garis/border ASCII.

```
func repeat(s string, n int) string {
	if n <= 0 {
		return ""
	}
	var b strings.Builder
	b.Grow(len(s) * n)
	for i := 0; i < n; i++ {
		b.WriteString(s)
	}
	return b.String()
}
```

2.6.9. printBox(title string, lines []string)

Membuat kotak header/instruksi CLI menggunakan karakter box-drawing.

```
func printBox(title string, lines []string) {
	maxLen := len(title)
	for _, ln := range lines {
		if len(ln) > maxLen {
			maxLen = len(ln)
		}
	}
	pad := 2
	width := maxLen + pad*2

	top := " ┌" + repeat("─", width) + "┐ "
	mid := " │ " + repeat(" ", pad) + title + repeat(" ", width-pad-len(title)) + " │ "

	fmt.Println(top)
	fmt.Println(mid)
	fmt.Println(" ├" + repeat("─", width) + "┤ ")
	for _, ln := range lines {
		row := " │ " + repeat(" ", pad) + ln + repeat(" ", width-pad-len(ln)) + " │ "
		fmt.Println(row)
	}
	fmt.Println(" └" + repeat("─", width) + "┘ ")
}
```

2.7. Rendering Board

2.7.1. queenLookup(solusi []int, n int) []int

Membuat mapping posisi queen per baris agar mudah dicek saat render (default -1 jika tidak ada queen).

```
func queenLookup(solusi []int, n int) []int {
	q := make([]int, n)
	for i := 0; i < n; i++ {
		q[i] = -1
	}
	if solusi == nil {
		return q
	}
	for r := 0; r < n && r < len(solusi); r++ {
		q[r] = solusi[r]
	}
	return q
}
```

2.7.2. regionColor(regionID int) ansi

Memilih warna berdasarkan regionID agar region berbeda terlihat kontras.

```
func regionColor(regionID int) ansi {
	if regionID < 0 {
		return "\x1b[90m" // gray
	}
	return regionFg[regionID%len(regionFg)]
}
```

2.7.3. formatCell(ch rune, regionID int, isQueen bool) string

Memformat isi sel: jika queen maka tampil # (bold putih), jika bukan queen maka tampil karakter region dengan warna region.

```
func formatCell(ch rune, regionID int, isQueen bool) string {
	if isQueen {
		return colorWrap("#", ansiBold, "\x1b[97m")
	}
	return colorWrap(string(ch), regionColor(regionID))
}
```

2.7.4. RenderBoardGrid(b *Queen, solusi []int) []string

Menghasilkan representasi board dalam bentuk grid ASCII lengkap (border dan separator). Jika solusi diberikan, queen akan direnderer sebagai # pada posisi yang sesuai.

```
func RenderBoardGrid(b *Queen, solusi []int) []string {
```

```
	N := b.n
	q := queenLookup(solusi, N)

	lines := make([]string, 0, N+2)

	cellW := 3
	top := "┌" + repeat("─", cellW)
	for i := 1; i < N; i++ {
		top += "┬" + repeat("─", cellW)
	}
	top += "┐"
	lines = append(lines, top)

	for r := 0; r < N; r++ {
		row := "│"
		for c := 0; c < N; c++ {
			ch := b.raw[r][c]
			reg := b.id[r][c]
			isQ := (q[r] == c)

			cell := formatCell(ch, reg, isQ)

			row += " " + cell + " "
			if c != N-1 {
				row += "│"
			}
		}
		row += "│"
		lines = append(lines, row)

		if r != N-1 {
			mid := "├" + repeat("─", cellW)
			for i := 1; i < N; i++ {
				mid += "┼" + repeat("─", cellW)
			}
			mid += "┤"
			lines = append(lines, mid)
		}
	}

	bot := "└" + repeat("─", cellW)
	for i := 1; i < N; i++ {
		bot += "┴" + repeat("─", cellW)
	}
	bot += "┘"
	lines = append(lines, bot)

	return lines
}
```

2.7.5. PrintBoardGrid(title string, b *Queen, solusi []int)

Mencetak judul dan grid board ke terminal (menggunakan hasil dari RenderBoardGrid).

```
func PrintBoardGrid(title string, b *Queen, solusi []int) {
	if title != "" {
		fmt.Println(colorWrap(title, ansiBold))
	}
	for _, ln := range RenderBoardGrid(b, solusi) {
		fmt.Println(ln)
	}
}
```

2.8. go.mod dan go.sum

2.8.1. go.mod mendefinisikan module Go dan versi Go yang digunakan

2.8.2. go.sum checksum dependensi module

3. Tangkapan Layar Input dan Output

Input	Output
Input1.txt	
AAABBCCCD ABBBBCECD ABBBDCECD AAABDCCCD BBBBDDDDD FGGGDDHDD FGIGDDHDD FGIGDDHDD FGGGDDHHH 	

Input2.txt

AAABBC
ADDBBC
ADDEEC
AFFEEC
AFFBBC
AAABBC

BOARD INPUT

A	A	A	B	B	C
A	D	D	B	B	C
A	D	D	E	E	C
A	F	F	E	E	C
A	F	F	B	B	C
A	A	A	B	B	C

BOARD RESULT

#	A	A	B	B	C
A	D	#	B	B	C
A	D	D	E	#	C
A	#	F	E	E	C
A	F	F	#	B	C
A	A	A	B	B	#

Waktu pencarian: 0 ms
Banyak kasus yang ditinjau: 3516 kasus

Input3.txt

AAABBCC
AEEBBCC
AEEBDDC
AFFFDCC
AGGGDDC
AGGGDCC
AGGGDCC

BOARD INPUT

A	A	A	B	B	C	C
A	E	E	B	B	C	C
A	E	E	B	D	D	C
A	F	F	F	D	C	C
A	G	G	G	D	D	C
A	G	G	G	D	C	C
A	G	G	G	D	C	C

BOARD RESULT

#	A	A	B	B	C	C
A	E	E	B	#	C	C
A	#	E	B	D	D	C
A	F	F	#	D	C	C
A	G	G	G	D	#	C
A	G	#	G	D	C	C
A	G	G	G	D	C	#

Waktu pencarian: 2 ms
Banyak kasus yang ditinjau: 70924 kasus

Input4.txt

AAABBCCCD
AEEBBCCCD
AEEBBDCCD
AFFFDCCCD
AGGGDDDHD
AGIIDDDHD
AGIIDDDHD
AGGGDDDHD
AAAGGGHHD

BOARD INPUT

A	A	A	B	B	C	C	C	D
A	E	E	B	B	C	C	C	D
A	E	E	B	B	D	C	C	D
A	F	F	F	D	C	C	C	D
A	G	G	G	D	D	D	H	D
A	G	I	I	D	D	D	H	D
A	G	I	I	D	D	D	H	D
A	G	G	G	D	D	D	H	D
A	A	A	G	G	G	H	H	D

BOARD RESULT

A	A	A	B	#	C	C	C	D
A	#	E	B	B	C	C	C	D
A	E	E	B	B	D	#	C	D
A	F	#	F	D	C	C	C	D
#	G	G	G	D	D	D	H	D
A	G	I	I	D	D	D	#	D
A	G	I	#	D	D	D	H	D
A	G	G	G	D	D	D	H	#
A	A	A	G	G	#	H	H	D

Waktu pencarian: 2445 ms
Banyak kasus yang ditinjau: 180282021 kasus

Input5.txt

AAABBCCCDD
AEEBBCCCDD
AEEBBCCCDD
AFFFDCCCDD
AGGGDDDHHH
AGIIIDDHHH
AGIIIDDHHH
AGGGDDDHHH
AAAGGGHHHH
JJJGGGHHHH

BOARD INPUT

A	A	A	B	B	C	C	C	D	D
A	E	E	B	B	C	C	C	D	D
A	E	E	B	B	C	C	C	D	D
A	F	F	F	D	C	C	C	D	D
A	G	G	G	D	D	D	H	H	H
A	G	I	I	I	D	D	H	H	H
A	G	I	I	I	D	D	H	H	H
A	G	G	G	D	D	D	H	H	H
A	A	A	G	G	G	H	H	H	H
J	J	J	G	G	G	H	H	H	H

NO SOLUTION
Waktu pencarian: 301403 ms
Banyak kasus yang ditinjau: 10000000000 kasus

4. Pranala Repository

- Link GitHub : https://github.com/agathatatiana936/Tucil1_13524008

5. Pernyataan Tidak Melakukan Kecurangan

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (Generative AI), melainkan hasil pemikiran dan analisis mandiri.

Agatha Tatianingseto - 13524008

6. Lampiran

No	Poin	Ya	Tidak
1	Program berhasil di kompilasi tanpa kesalahan	v	
2	Program berhasil di jalankan	v	
3	Solusi yang diberikan program benar dan mematuhi aturan permainan	v	
4	Program dapat membaca masukan berkas .txt serta menyimpan solusi dalam berkas .txt	v	
5	Program memiliki Graphical User Interface (GUI)		v
6	Program dapat menyimpan solusi dalam bentuk file gambar	v	